Contador Serverless AWS

Bruno Ruan Lima Da Silva

David Bezerra Da Silva

Natan Ferreira Dos Santos

Gustavo Ramalho Matioli

Introdução e Apresentação do Problema

Uma *startup* do segmento de marketing planeja lançar uma campanha para um novo produto e, para isso, desenvolveu uma *landing page*, uma página web estratégica onde os usuários podem demonstrar interesse no projeto. Diante do lançamento, a empresa precisa quantificar esse interesse em tempo real para avaliar a aceitação do mercado e decidir se deve dar continuidade à ideia ou reformular o produto.

No entanto, o caráter inédito da campanha traz uma incerteza severa quanto ao volume de acessos. Caso o site receba um pico inesperado de tráfego, a infraestrutura tradicional pode sofrer com problemas de latência, travamentos e lentidão na resposta do servidor.

A necessidade de resolver esse problema é crítica e baseia-se diretamente na experiência do usuário. Enquanto métricas do Google (2024) apontam que 53% dos visitantes abandonam sites que demoram mais de três segundos para carregar, essa mesma exigência de performance se aplica às interações ativas. Se, ao clicar no botão de interesse, o back-end demorar a processar a requisição devido à sobrecarga, a interface parecerá travada e pouco responsiva. Considerando que o Nielsen Norman Group mostra que os usuários formam uma opinião sobre a credibilidade de uma aplicação em frações de segundo (50 milissegundos), garantir que o registro do clique seja rápido, estável e invisível para o usuário é indispensável para não gerar frustração e inviabilizar a coleta de dados.

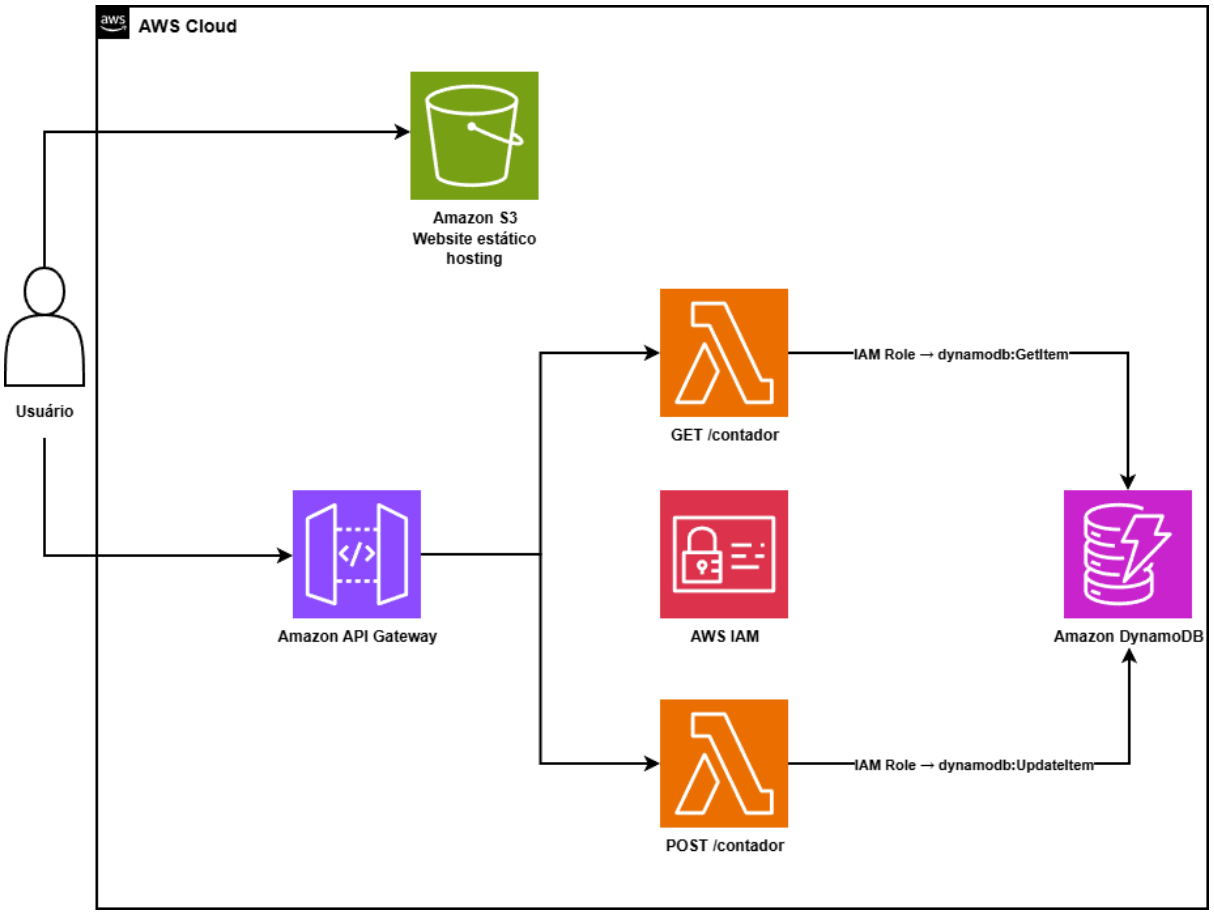
Como a Solução Resolve o Problema

Para mitigar os riscos de indisponibilidade e alto custo, a solução proposta consiste no desenvolvimento de um contador de cliques de interesse automatizado, baseado em arquitetura serverless (computação sem servidor).

Esse modelo resolve o problema ao alinhar perfeitamente a infraestrutura à demanda flutuante da campanha. Por ser altamente escalável, o sistema se adapta automaticamente

a qualquer volume de requisições, seja de dezenas ou de milhões de interações simultâneas, eliminando gargalos de lentidão e garantindo um tempo de resposta inferior ao limite crítico de 3 segundos. Além disso, a abordagem *serverless* garante o baixo custo, uma vez que a *startup* só pagará pelo tempo exato de processamento das requisições, evitando gastos com servidores ociosos caso o volume de cliques seja menor que o esperado.

Diagrama de Arquitetura da Solução



Modelagem Lógica da Solução

O funcionamento da solução segue um fluxo linear, síncrono e orientado a eventos. O ciclo de vida de uma requisição opera de forma simplificada através das seguintes etapas sequenciais:

1. **Acesso (Amazon S3):** O navegador carrega os arquivos estáticos da *landing page* diretamente do S3.

2. **Disparo:** O clique no botão gera uma requisição HTTP síncrona para o API Gateway.

3. **Recepção (Amazon API Gateway):** Valida CORS, aplica *throttling* básico e repassa o evento para a camada de computação.
4. **Execução & Segurança (AWS Lambda & IAM):** O Lambda é acionado e assume uma *Execution Role* do IAM com permissão restrita ao banco.
5. **Persistência (Amazon DynamoDB):** O Lambda executa o incremento atômico (`UpdateItem`) na tabela do DynamoDB.
6. **Retorno:** O resultado atualizado retorna pelo Lambda e API Gateway até o navegador para atualizar a tela.

Serviços Utilizados

1. Amazon S3 (Simple Storage Service)

Serviço de armazenamento de objetos projetado para armazenar e recuperar qualquer volume de dados com alta disponibilidade. No projeto, funciona como o repositório e servidor dos arquivos estáticos da *landing page* (HTML, CSS e JavaScript), configurado com o recurso de hospedagem de site estático (*Static Website Hosting*) para servir a interface diretamente ao navegador do usuário.

2. Amazon API Gateway

Porta de entrada gerenciada que intercepta as requisições HTTP originadas pelo navegador na *landing page*. É responsável por gerenciar o ciclo de vida da API, validar cabeçalhos obrigatórios de CORS (*Cross-Origin Resource Sharing*) para autorizar as chamadas vindas do S3 e aplicar políticas nativas de *Throttling* e *Rate Limiting* para proteção do back-end antes de rotear o evento à camada de computação.

3. AWS Lambda

Camada de computação *serverless* orientada a eventos que executa o back-end da aplicação sob demanda, sem necessidade de provisionamento de infraestrutura. Instanciado a partir do acionamento do Amazon API Gateway, processa a requisição e encaminha a instrução de incremento atômico diretamente para o banco de dados, eliminando custos com ociosidade de hardware.

4. Amazon DynamoDB

Banco de dados NoSQL de chave-valor totalmente gerenciado que atua como a persistência de dados do contador. Processa a operação `UpdateItem` originada pelo AWS Lambda no lado do servidor de forma atômica, o que garante a consistência das métricas sob alta concorrência e mantém o tempo de resposta na casa dos milissegundos.

5. **Amazon IAM (Identity and Access Management)**

Serviço de controle de acesso centralizado configurado para garantir o princípio do menor privilégio na comunicação entre os recursos. Define a função de execução (*Execution Role*) do AWS Lambda, concedendo permissão estrita e exclusiva para leitura e escrita (`UpdateItem`) na tabela específica do DynamoDB, bloqueando qualquer outra ação lateral não autorizada na infraestrutura.

Justificativa de Escolhas Técnicas

Os serviços escolhidos para a construção da arquitetura foram escolhidos seguindo rigorosamente as regras do negócio, as especificações técnicas de escalabilidade e as diretrizes do *AWS Well-Architected Framework*.

A arquitetura proposta pelo Amazon DynamoDB, Lambda e API Gateway foram selecionados pela característica em comum de serem serviços *Serverless*. O principal motivo para a decisão foi a imprevisibilidade de tráfego que terá na página do contador de acessos, essa abordagem garante com que a solução tenha:

- Elasticidade Total: A infraestrutura escala horizontalmente de forma automática, respondendo de forma eficiente tanto a cenários de ociosidade quanto a picos de milhões de requisições simultâneas.
- Eficiência de Custo (Pay-per-Use): Alinhado ao pilar de Otimização de Custos do *Well-Architected*, elimina-se o custo de capacidade ociosa. A cobrança é efetuada estritamente pelo volume de execuções e processamento consumido.
- Core Business: Por serem serviços totalmente gerenciados, a responsabilidade pela manutenção da infraestrutura, *patching* de segurança e alta disponibilidade fica a cargo da AWS, permitindo foco exclusivo na lógica de negócio.

Outras Abordagens

No processo de decisão sobre a estrutura da infraestrutura do projeto, foram consideradas duas abordagens alternativas para resolver o problema. A seguir, apresenta-se a análise detalhada dessas opções e os motivos de suas desqualificações.

Abordagem 1: Arquitetura com Instâncias EC2 e Banco Relacional (RDS)

Nesta abordagem, utilizaria-se uma máquina virtual (Amazon EC2) rodando uma API conectada a um banco de dados relacional (Amazon RDS com PostgreSQL ou MySQL).

A ideia foi descartada devido ao custo de ociosidade. Como o tráfego da aplicação é imprevisível, manter uma instância no modelo sob demanda geraria um custo fixo elevado mesmo em períodos de baixo acesso. Para uma tarefa de baixa complexidade computacional, como incrementar um contador (+1), essa abordagem representaria um gasto desnecessário.

Além disso, para garantir a escalabilidade contra picos repentinos, seria necessária a configuração de *Auto Scaling Groups* e *Load Balancers (ALB)*. O tempo de inicialização (*boot*) de uma nova instância EC2 (mensurado em minutos) é lento demais para absorver picos repentinos de cliques, o que geraria gargalos. Por fim, haveria uma alta sobrecarga operacional, pois a equipe seria responsável por atualizar o sistema operacional da máquina e gerenciar o pool de conexões do banco de dados, elevando a complexidade do projeto sem necessidade real.

Abordagem 2: Arquitetura Baseada em Containers (AWS ECS/EKS com Fargate)

A segunda abordagem avaliada baseava-se na utilização de containers Docker gerenciados pelo AWS ECS com AWS Fargate (modelo *serverless* para containers) hospedando uma API para processar os cliques.

Esta opção foi descartada por resultar em subutilização de recursos para tarefas simples. Um contador de cliques executa uma instrução extremamente simples e rápida. Provisionar e inicializar um container completo para rodar essa função geraria uma sobrecarga de processamento desnecessário.

Ademais, o modelo de cobrança do AWS Fargate é baseado no tempo de execução do container ativo (vCPU e memória alocada por segundo), cobrando mesmo quando o container está ocioso aguardando uma requisição. Em contrapartida, o AWS Lambda cobra estritamente pelo milissegundo de processamento da função.

Sob a perspectiva de escalabilidade, gerenciar a elasticidade dos containers exigiria o monitoramento de métricas como CPU e Memória, e o tempo de reação seria insuficiente caso o tráfego exploda em questão de segundos.

Pilares Atendidos e Funcionamento dos Serviços

A arquitetura da solução foi projetada seguindo as diretrizes e boas práticas do AWS Well-Architected Framework, garantindo uma infraestrutura em nuvem otimizada, segura e eficiente. A integração entre os componentes atende rigorosamente aos pilares de eficiência de desempenho, confiabilidade e segurança.

1. Eficiência de Desempenho e Escalabilidade

O pilar de eficiência de desempenho é atendido através do uso de serviços *serverless* orientados a eventos. Na camada de *front-end*, o Amazon S3 entrega os arquivos estáticos da *landing page* diretamente via recurso de hospedagem de site estático, garantindo o carregamento imediato da interface.

Na camada de *back-end*, o Amazon API Gateway gerencia a entrada das requisições HTTP de forma escalável e aciona o AWS Lambda. O Lambda escala horizontalmente de forma automática conforme o volume de interações cresce, o que elimina gargalos de processamento e trata requisições concorrentes sem a necessidade de pré-provisionamento de servidor.

Complementando esse fluxo, o Amazon DynamoDB fornece a persistência rápida do estado do contador. O banco de dados processa gravações e incrementos atômicos por meio da operação `UpdateItem` na casa dos milissegundos de um dígito. Essa combinação de serviços garante que o sistema registre a intenção do usuário e retorne a resposta final ao navegador em tempo hábil, suportando variações imprevisíveis de tráfego.

2. Confiabilidade e Resiliência

A resiliência da solução apoia-se na natureza nativamente distribuída da infraestrutura da AWS. O armazenamento dos arquivos estáticos no Amazon S3 conta com alta disponibilidade e alta durabilidade, replicando os dados automaticamente entre múltiplas Zonas de Disponibilidade (*Availability Zones*).

Na camada de aplicação e persistência, o Amazon API Gateway, o AWS Lambda e o Amazon DynamoDB também operam de forma totalmente gerenciada e resiliente,

distribuindo a carga de processamento e a armazenagem de dados entre múltiplas Zonas de Disponibilidade. Caso ocorra uma falha em um dos *data centers* subjacentes, a infraestrutura da AWS redireciona o tráfego instantaneamente, assegurando que o contador continue operando sem interrupções e sem a necessidade de intervenção manual ou configuração de *clusters*.

3. Segurança

A segurança é implementada em profundidade através de controles em camadas para conter acessos indevidos e aplicar o princípio do menor privilégio. Na entrada do *back-end*, o Amazon API Gateway atua como uma camada de proteção periférica que isola a lógica do Lambda. Ele valida os cabeçalhos de CORS (*Cross-Origin Resource Sharing*) para autorizar apenas requisições originadas pelo domínio da *landing page* e aplica limites de *Throttling* e *Rate Limiting* para conter abusos de requisições, protegendo a computação e evitando custos imprevistos com execuções excessivas.

No repositório do *front-end*, o Amazon S3 limita o acesso estritamente à leitura dos arquivos públicos estáticos necessários para a renderização do site, mantendo as permissões de escrita e alteração restritas.

Por fim, o princípio do menor privilégio é gerenciado pelo AWS IAM. A *Execution Role* atribuída ao AWS Lambda concede exclusivamente a permissão técnica necessária para executar a ação `UpdateItem` na tabela específica do Amazon DynamoDB, bloqueando qualquer outra ação lateral não autorizada no banco de dados ou em outros recursos do ambiente.

Custo da Infraestrutura da Solução

aws

Contact your AWS representative: Contact Sales

Export Date: 07/23/2026 **Language:** English

Estimate url

Estimate summary

Upfront cost	Monthly cost	Total 12 months cost
0.00 USD	1.56 USD	18.72 USD Includes upfront cost

Detailed Estimate

Name	Group	Region	Upfront cost	Monthly cost
Amazon API Gateway	-	US East (N. Virginia)	0.00 USD	1.00 USD

Status -

Description: A calculadora projeta um custo de 1 dólar para 1 milhão de requisições. O API Gateway oferece 1 milhão de requisições gratuitas por mês no Free Tier. Custo real de 0 dólares.

Config summary HTTP API requests units (millions), Average size of each request (34 KB), REST API request units (exact number), Cache memory size (GB) (None), WebSocket message units (thousands), Average message size (32 KB), Requests (1 per month)

Name	Group	Region	Upfront cost	Monthly cost
Amazon Simple Storage Service (S3)	-	US East (N. Virginia)	0.00 USD	0.00 USD

Status -

Description: A aplicação possui um frontend de 3,63KB, e o S3 fornece 5GB gratuitamente.

Config summary S3 Standard storage (0.0005 GB per month)

Name	Group	Region	Upfront cost	Monthly cost
AWS Lambda	-	US East (N. Virginia)	0.00 USD	0.00 USD

Status -

Description: O Lambda possui um Free Tier vitalício de 1 milhão de requisições e 400gb/s de computação por mês, no qual se encaixa totalmente o projeto considerando um máximo de 1milhão de acessos.

Config summary Invoke Mode (Buffered), Architecture (x86), Architecture (x86), Number of requests (1 million per month), Amount of ephemeral storage allocated (512 MB)

Name	Group	Region	Upfront cost	Monthly cost
Amazon DynamoDB	-	US East (N. Virginia)	0.00 USD	0.56 USD

Status -

Description: 1gb de armazenamento, com um máximo de 1 milhão em leituras por mês. O custo real será zero, já que as configurações atuais se encaixam totalmente no free tier do DynamoDB.

Config summary Table class (Standard), Average item size (all attributes) (1 KB), Write reserved capacity term (1 year), Read reserved capacity term (1 year)

Acknowledgement

AWS Pricing Calculator provides only an estimate of your AWS fees and doesn't include any taxes that might apply. Your actual fees depend on a variety of factors, including your actual usage of AWS services. Learn more

Referências:

https://www.thinkwithgoogle.com/intl/pt-br/estrategias-de-marketing/apps-e-mobile/como-acompanhar-os-novos-benchmarks-da-sua-industria-para-velocidade-de-paginas-mobile/

https://www.nngroup.com/articles/five-seconds-website/

https://aws.amazon.com/pt/pm/lambda/

https://aws.amazon.com/pt/dynamodb/

https://aws.amazon.com/pt/iam/

https://aws.amazon.com/api-gateway/pricing/

https://aws.amazon.com/pt/api-gateway/

https://calculator.aws/#/estimate?id=7742ca74d8f9d07fd1343dc351146bca0ec3211a